GI ENDOSCOPY IMAGE ANALYSIS SYSTEM

Classification, Polyp Segmentation & AI-Generated Explanation Pipeline

Project Report

Built and trained on Google Colab (Tesla T4 GPU)

Seasons of Code, IIT Bombay

Final Project Report — AI for Healthcare

Shashank Choudhary (Roll No. 25B0371)

Mentor: Viren Mehta

1. Project Overview

This project is a two-stage deep learning pipeline for gastrointestinal (GI) endoscopy images, combined with an AI agent that turns the raw model output into a plain-language explanation. It was built and run entirely in Google Colab using a Tesla T4 GPU.

The pipeline works in three steps:

- Classification — a YOLOv8 model sorts an endoscopy frame into one of 8 clinical categories (e.g. polyps, esophagitis, normal tissue types).
- Conditional segmentation — if the classifier finds a polyp, a second YOLOv8 model outlines its exact boundary and measures how much of the image it covers.
- AI explanation — an LLM agent (Groq/Llama 3.3, via LangChain) reads the model's numbers and writes a plain-language explanation for the user, with a live web-search tool for anything current (e.g. treatment guidelines).

The finished system was wrapped in a Streamlit web app and exposed publicly through an ngrok tunnel so it could be tested from a browser.

Tools & Technology Stack

Component	Tool / Library
Environment	Google Colab (Tesla T4 GPU)
Classification model	YOLOv8n-cls (Ultralytics)
Segmentation model	YOLOv8n-seg (Ultralytics)
Mask → label conversion	OpenCV (contour detection)
AI explanation agent	LangChain + Groq (Llama 3.3 70B)
Web search grounding	Serper API (Google Search)
Weight persistence	Google Drive
Web app / deployment	Streamlit + ngrok

2. Dataset Details

2.1 Classification dataset — Kvasir v2

Kvasir v2 is a labelled dataset of GI endoscopy images across 8 classes covering both anatomical landmarks and pathological findings: dyed-lifted-polyps, dyed-resection-margins, esophagitis, normal-cecum, normal-pylorus, normal-z-line, polyps, and ulcerative-colitis.

Total images	Classes	Images / class
8,000	8	1,000

The full set was split per class (70% / 15% / 15%) using a fixed random seed, giving a balanced split across all classes:

Split	Images / class	Total images
Train	700	5,600
Validation	150	1,200
Test	150	1,200

2.2 Segmentation dataset — Kvasir-SEG

Kvasir-SEG is a polyp-specific dataset containing endoscopy images paired with pixel-level segmentation masks. Since YOLOv8-seg needs polygon labels rather than mask images, each binary mask was converted using OpenCV: contours were extracted from the mask, simplified, normalized to 0–1 image coordinates, and written out as single-class ("polyp") YOLO polygon label files. Every image-mask pair matched successfully with none skipped. The labelled set was then split 85% train / 15% validation.

3. How It Was Built

3.1 Classification model

A YOLOv8n-cls (nano) model was chosen as a fast, lightweight baseline for the 8-way classification task, trained for 30 epochs at 224×224 resolution with a batch size of 32 and early stopping (patience 5 epochs).

3.2 Segmentation model

A YOLOv8n-seg (nano) model was trained on the converted Kvasir-SEG polygon labels for 50 epochs at 224×224, batch size 16, with early stopping (patience 8 epochs). Its only job is to outline polyp boundaries — it is never run on images the classifier didn't already flag as polyps.

3.3 Two-stage inference pipeline

Rather than always running both models, the pipeline is conditional and mirrors how a clinician would triage a frame:

- Every image is first passed through the classifier.
- If the top prediction is "polyps" and confidence ≥ 50%, the segmentation model runs on that same image.
- If a mask is found, its pixel area is measured against the full image to compute a coverage percentage, which is bucketed into small (<1%), medium (1–5%) or large (>5%).

- If the classifier says "polyps" but the segmentation model can't confidently outline a boundary, the system still returns a note explaining that instead of silently failing.
- For every other class, segmentation is skipped entirely — saving compute and avoiding meaningless boundary boxes on healthy tissue.

This structured record (predicted class, confidence, polyp coverage, size bucket) is exactly what gets fed to the explanation agent — nothing is re-interpreted by the LLM, it only explains numbers of the vision models already produced.

3.4 AI explanation agent

A LangChain agent wraps a Groq-hosted Llama 3.3 70B model with a single tool: a Google Search wrapper (via Serper API) for current medical information such as treatment guidelines. The agent's system prompt is built dynamically per result and is deliberately constrained: it must treat the model's classification and confidence as given (not second-guess them), must never search for anything about the user's personal severity or prognosis, must cite sources when it does search, and must always close with a reminder that this is not a diagnosis.

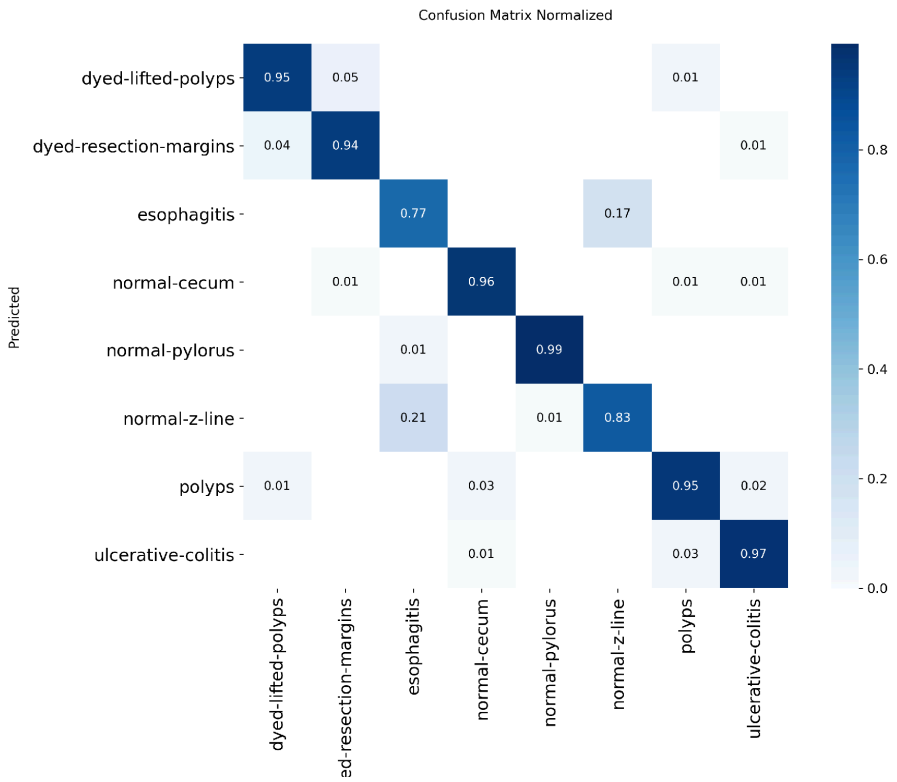
The whole thing was packaged into a Streamlit app (app.py) that loads both YOLO models and the agent once (cached), lets a user upload an image, shows the classification, the segmentation overlay if relevant, and a chat box for follow-up questions grounded in that same result.

4. Results

4.1 Classification performance

Evaluated on the held-out test set (1,200 images, never seen during training):

Top-1 accuracy	Top-5 accuracy	Test images
92.1%	100%	1,200



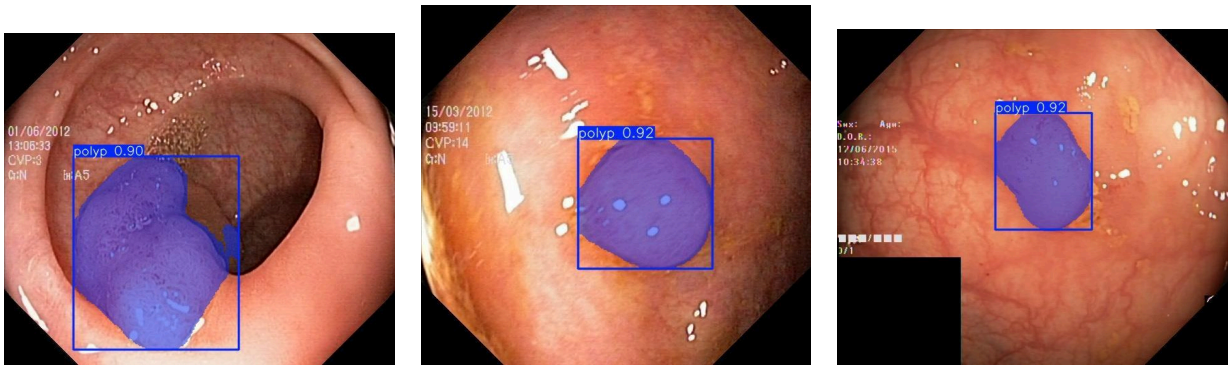
Normalized confusion matrix on the test set — the diagonal shows how consistently each class is identified correctly.

4.2 Segmentation performance

Evaluated on the segmentation validation set (150 images, 158 annotated polyp instances) at the end of training:

Metric	Precision	Recall	mAP50	mAP50-95
Box (localization)	0.934	0.873	0.939	0.703
Mask (pixel outline)	0.947	0.886	0.942	0.691

mAP50 above 0.93 for both the bounding box and the pixel mask means the model is reliably finding and outlining polyps it's shown; the mAP50-95 figures (~0.70) reflect the stricter standard of requiring a tight pixel-level match, which is normal for a small, fast "nano" model on a modest-sized medical dataset.



Sample polyp segmentation outputs on validation images — outline and confidence drawn by the trained YOLOv8n-seg model.

4.3 End-to-end pipeline output

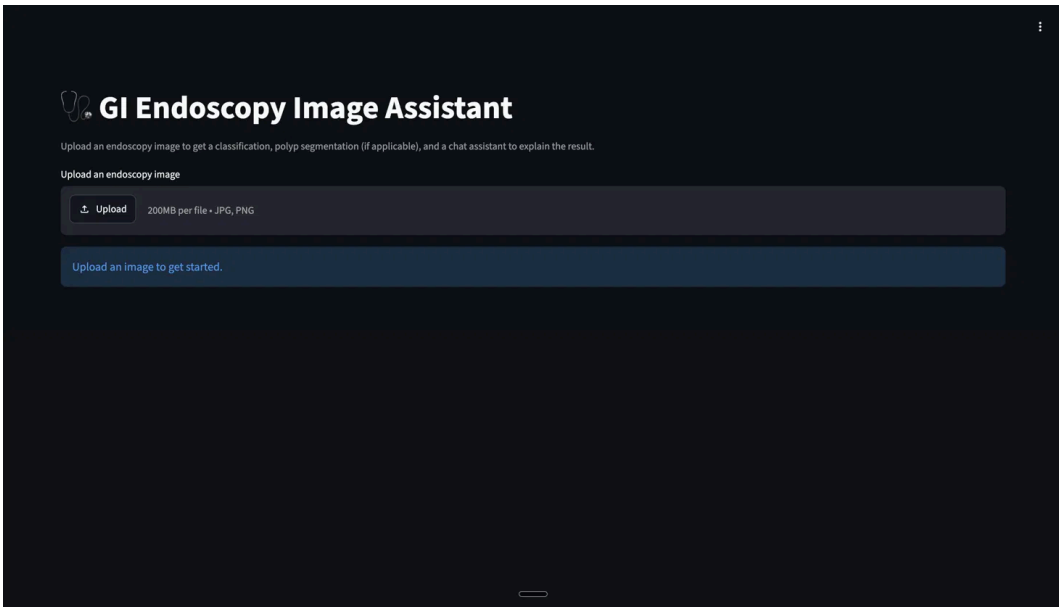
Two representative runs through the full pipeline, exactly as produced by the system:

Input image	Predicted class	Confidence	Segmentation result
Polyp case	polyps	100%	1 polyp detected, 9.04% of image area → "large"
Healthy case	normal-cecum	99.67%	Skipped (no polyp predicted)

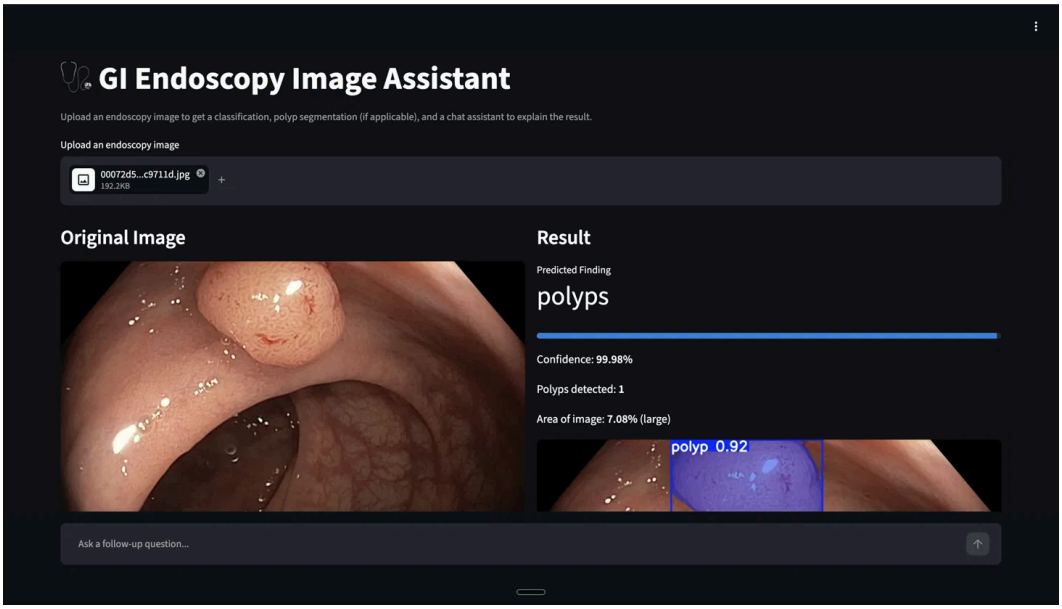
On the polyp case, the explanation agent's opening response accurately restated the finding in plain language, correctly flagged it as worth a doctor's attention without overstating certainty, and closed with a not-a-diagnosis disclaimer as instructed. When asked a follow-up requiring current information ("latest treatment guidelines for colon polyps"), it correctly triggered the web-search tool rather than answering from memory; when asked a question about personal severity, it correctly answered directly from the model result instead of searching — matching the intended tool-use behaviour.

4.4 Application Interface

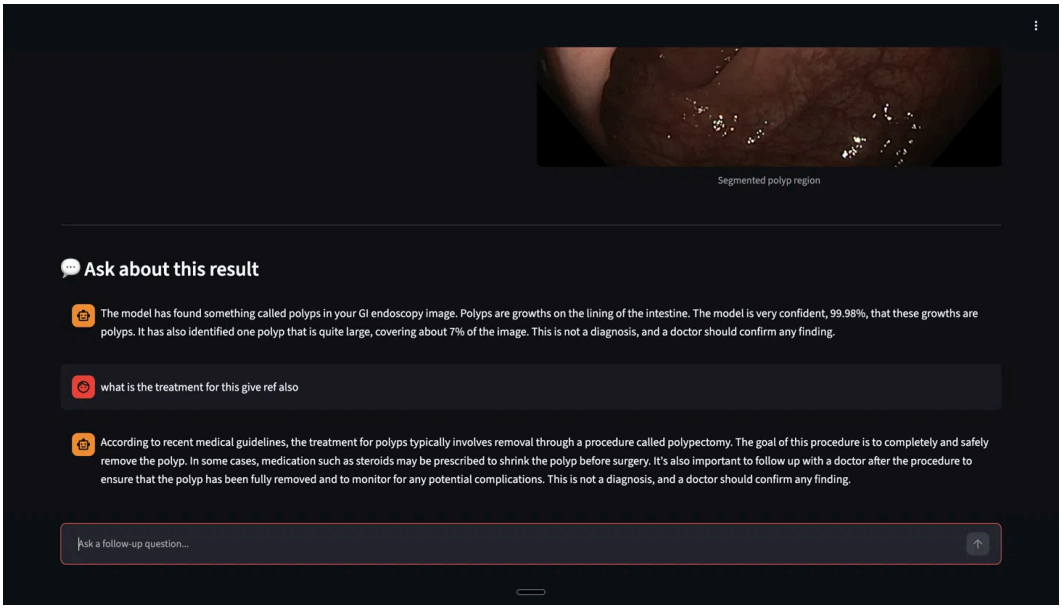
The deployed Streamlit app (GI Endoscopy Image Assistant) lets a user upload an image, view the classification and segmentation result side by side, and ask follow-up questions in a chat box grounded in that same result.



Landing screen — the user uploads an endoscopy image to get started.



Classification and segmentation result for an uploaded polyp image, with confidence, polyp count, and area coverage.



Segmented polyp region with the follow-up chat assistant explaining the finding and, on request, current treatment guidelines with sources.

5. Problems Faced & What Was Changed

Problem	Fix / decision made
Kvasir dataset host's SSL certificate couldn't be verified during download	Used --no-check-certificate on wget rather than abandoning the source
Kvasir-SEG masks aren't in YOLO's polygon label format	Wrote a custom OpenCV contour-extraction step to convert every mask into normalized polygon .txt labels
Colab runtime resets and wipes local files between sessions	Mounted Google Drive and copied both best.pt checkpoints there so training didn't need to be repeated
Running segmentation on every image wastes compute and produces meaningless output on non-polyp frames	Made segmentation conditional — it only runs when the classifier predicts "polyps" above a confidence threshold
Preferred Groq tool-use model (llama3-groq-70b-8192-tool-use-preview) was unreliable / unavailable at deploy time	Added a try/except fallback in the app so it automatically drops to llama-3.3-70b-versatile if the preferred model fails
Needed to expose the Colab-hosted Streamlit app to a browser for testing	Initially set up localtunnel, then standardized on ngrok with an authenticated tunnel for a stable public URL
Risk of the LLM inventing or second-guessing clinical findings	Constrained the system prompt so the agent treats the model's output as ground truth and only uses web search for general medical/current-guideline questions, never personal severity

6.How It Worked Out

The final system reliably classifies GI endoscopy frames into 8 categories at 92.1% top-1 accuracy, and when a polyp is found, the segmentation stage outlines it with a mask mAP50 of 0.942 strong performance for two lightweight "nano" YOLOv8 models trained quickly on a single T4 GPU. The conditional pipeline design keeps segmentation focused only on relevant cases, and the LLM layer turns raw numbers into a grounded, appropriately hedged explanation rather than a diagnosis. The whole pipeline upload, classify, segment, explain, chat runs live through the deployed Streamlit app.

Because both models are the smallest variants in their families, there is a clear next step if higher accuracy is needed: re-training with yolov8s-cls / yolov8s-seg (already anticipated in the original code comments) on the same splits.